Last modified on

Sports Data Soccer API - Remote Player Runs Feed (MA59)

This feed provides details of each run that happens in the game.

Overview

The Soccer Remote Player Runs feed provides all runs by all players across both teams, as they occurred in chronological order. This feed will list each run that happens in the game. Each run will have a start and stop time, player id, start frame, end frame (same frame id as MA49), a category, multiple event references and multiple qualifiers. The data is based on events and remote tracking.

> **Required Compliance:**
> To comply with our API security and best practice logic for your API calls please refer to our FAQs. We also recommend that you take a look at our API Overview and Authentication Guide. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

UPDATED	POLLING FREQUENCY							
	TIER 15	TIER 12+14	TIER 4	TIER 13	TIER 8+9+10+11	TIER 6+7	TIER 5	TIER 1+2+3
Updated on every stop in play during the game, including for the following types of events: goal, shot at goal, assist, free kick, offside given, cards, start of the half, end of the half, corner, substitution, lineup **Note:** There will always be a gap of at least 30 seconds between files unless a goal is scored	See Tier 13	Post-match	n/a	Post-match	n/a	n/a	n/a	n/a

---

Remote Player Runs (MA59) Guide

USAGE	FEE
GET information about shape and formation of each player associated with a specific MFL entity UUID, based on “in venue” data *Required:* query parameter and UUID for the entity (such as a fixturre)	/s

> If your request URL is as follows (does not contain query criteria), the feed returns an error:
> `/soccerdata/remoteplayerruns/{outletAuthKey}`

Query Parameters And Example Request Parameters

This feed also makes use of the Global query parameters, and you can combine them with the following query parameters (some of which can be used in conjunction with each other). However, you must query the `/remoteplayerruns` feed by an entity such as a fixture UUID and you can combine query parameters in your requests.

Legacy IDs: You can use the `fx` parameters to filter by legacy IDs.

> **How to use query parameters and make successful requests:**
> In our guides, we use HTTPS syntax for our example `GET` requests (rather than cURL, for example). All URIs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the main guide and API Overview and Authorisation guide.
>
> - Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` (Outlet Authentication Key), query parameters for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
> - Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
> - Use the required `_` (underscore) for any parameters listed with this prefix.
> - Remove any placeholder value curly braces `{}` from your calls (we include these as an example only).
> - If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
`fixtureUuid`	Get **In-venue player shape** data by passing the fixture/match UUID inline URL path. `GET https://api.performfeeds.com/soccerdata/remoteplayerruns/{outletAuthKey}/{fixtureUuid}?_fmt={dataFormat}&_rt={mode}`
`fx`	GET **In-venue player shape** data by the specified **fixture**. Pass the match ID to the `fx` parameter.

Query parameter	Value and description
	`GET https://api.performfeeds.com/soccerdata/remoteplayerruns/{outletAuthKey}?fx={fixtureUuid}&_fmt={dataFormat}&_rt={mode}`
`lcl`	GET In-venue player shape data by passing `_lcl` parameter. `GET https://api.performfeeds.com/soccerdata/remoteplayerruns/{outletAuthKey}?fx={fixtureUuid}&_fmt={dataFormat}&_rt={mode}&_lcl={language code}`

Sample Calls

- XML
- JSON

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

▸<remotePlayerRuns> (XML only)

▸<matchInfo>

▸<description>

▸<sport>

▸<ruleset>

▸<competition>

▸<country>

▸<tournamentCalendar>

▸<stage>

▸<series>

▸<contestants> (XML only)

▸<contestant>

▸<country>

▸<venue>

▸<liveData>

▸<matchDetails>

▸<periods> (XML only)

▸<period>

▸<scores>

▸<ht>

▸<ft>

▸<total>

▸<goal>

▸<card>

▸<substitute>

▸<matchDetailsExtra>

▸<attendance>

▸<matchOfficials> (XML only)

▸<matchOfficial>

▸<runByRun>

▸<run>

▸<labels>

▸<label>

▸<qualifiers>

▸<qualifier>

Definitions Repeated Throughout The Feed

There are some definitions repeated throughout the feed and are defined here.

- `id` (int): a unique identifier (unique per Phase per game)
- `startTime` (float): the start time of the Phase within the current period in seconds
- `endTime` (float): the end time of the Phase within the current period in seconds
- `startFrame` (int): the start frame of the Phase as seen in the Opta Vision in-venue or remote feeds.
- `endFrame` (int): the end frame of the Phase as seen in the Opta Vision in-venue or remote feeds
- `startX` (float): the x-coordinate in the first frame of the Run
- `startY` (float): the y-coordinate in the first frame of the Run
- `endX` (float): the x-coordinate in the final frame of the Run
- `endY` (float): the y-coordinate in the final frame of the Run

Feed Definitions

Below are brief explanations of, entries in the MA59

- `start`
  - `startTime/startFrame`
  - `startX/startY`
  - `firstTouchPlayerId, firstTouchEventId` (int): the event_id of the first Opta Touch Event during the phase, and the corresponding player_id for this touch event. Note that not all phases have to include a touch event/player.
  - `initiatorPlayerId, initiatorEventId` (int): Defined in detail here. Note that not all phases have to include an initiator player/event.
- `end`
  - `endTime/endFrame`
  - `endX/endY`
- `labels`
  - `master` (str): one of the 10 In Possession or 5 Out Of Possession Run labels. if multiple labels are present, the highest ranking in the hierarchy is chosen
  - `secondary` (array of strs): all of the possible 10 In Possession or 5 Out Of Possession Run labels. can be multiple
- `qualifiers`
  - `relatedEvents` (array of ints): The Opta event_ids of all events that happened during the Run
  - `speed`
    - `threshold` (str): The pace category of the Run, either Sprint (>= 7m/s) or High speed (>=5.5m/s)
    - `max` (float): The maximum instantaneous (i.e. frame-to-frame) speed (in metres per second) of the Run
- `pressure` (str): the Pressure category of the Run, either `high`, `medium` or `low`
- `pass`
  - `target` (bool): whether the runner was the target of a pass during their run
  - `received` (bool): whether the runner successfully received a pass during their run
- `expectedReceiver`
  - `start` (float): the first Expected Receiver (xR) value predicted for the runner during any teammate possession during their run
  - `end` (float): the last xR value predicted for the runner during any teammate possession during their run
  - `max` (float): the maximum xR value predicted for the runner during any teammate possession during their run
- `expectedThreat` :as for expectedReceiver above, but concerning Expected Threat (xT)
- `expectedPass` :as for expectedReceiver above, but concerning Expected Pass completion (xP)
- `increasesAvailability` (bool): whether the runner increased their Expected Receiver value by at least 0.3 xR and to a minimum of at least 0.8 xR during a single teammate possession
- `dangerous` (bool): whether the runner increased their Expected Threat value by at least 0.05 xT and to a minimum of at least 0.2 xT during a single teammate possession
- `runDuration` (float): the duration (in seconds) of the Run
- `runDistance` (float): the distance (in metres) of the Run; calculated by summing the frame-to-frame distance, not just the linear distance from the start to the end of the Run
- `runDirection` (float): the direction (in degrees) of the run, where 0° represents a directly vertical run (i.e. towards the opponent's goalline)
- `runStraightness` (float): directness of Run defined as tortuosity (i.e. total distance covered in the curved run divided by linear distance between start and end points of the run)

- `runEndsInBox` (bool): whether the Run ends in opposition penalty area
- `runStartsBehindBall` `runEndsBehindBall` (bool): whether the Run starts / ends behind the ball
- `runStartsInFrontOfBall` `runEndsInFrontOfBall` (bool): whether the Run starts / ends ahead of the ball
- `runAwayFromBall` (bool): if distance of the runner to the ball is higher at the end of the Run than at the start
- `runTowardOppositionGoal` (bool): if distance of the runner to the opposition goal is lower at the end of the Run than at the start
- `closestDistanceToBallCarrier` (float): minimum distance (in metres) to any teammate with possession of the ball during the run
- `defensiveLineBroken` `midfieldLineBroken` (bool): whether the Runner runs through the opposition defensive / midfield line
- `runEndsBetweenLines` (bool):whether the Run starts / ends between the opposition defensive and midfield lines
- `runFollowedByTeamShot` `runFollowedByTeamGoal` (bool):whether there is a shot / goal by the team of the runner within 10 seconds of their Run starting